

Book 5: Scalable Financial System Design

Payment Switches, Distributed Ledgers & Resilient Architecture

IDFC FIRST Bank | Strategic Projects Developer (3+ Years Experience)

Personalized Hinglish Interview Preparation System • Production Ready

Financial System Design Fundamentals & The 10-Step Interview Framework

1. Why This Matters

For a **Developer with 3+ years experience** applying to IDFC FIRST Bank's Strategic Projects, the System Design round is often the primary filter separating junior coders from true backend engineers. In this round, the interviewer provides an intentionally vague prompt (e.g., *"Design a UPI Payment Switch"* or *"Design a High-Throughput Transaction Ledger"*). They evaluate whether you panic, jump immediately to drawing boxes, or systematically guide the interview through requirements, scale calculations, data consistency, idempotency, and failure mitigation.

2. Prerequisites

- Client-server architecture, HTTP/HTTPS, REST, and gRPC.
 - Relational databases (PostgreSQL/MySQL), NoSQL, Caching (Redis), and Message Brokers (Kafka/RabbitMQ).
 - Basic networking: DNS, Load Balancers (L4 vs L7), Reverse Proxies.
-

3. Concept: The 10-Step Banking System Design Framework

flowchart TD

```
S1["1. Clarify Requirements (Functional & Non-Functional)"] --> S2["2. Scale Estimation (TPS, Storage, Bandwidth)"]
S2 --> S3["3. API Design (Idempotent Contracts, REST/gRPC)"]
S3 --> S4["4. Data Model & Storage Choice (Relational, ACID, Polyglot)"]
S4 --> S5["5. High-Level Architecture (Gateway, Microservices, DB)"]
S5 --> S6["6. Core Component Deep Dive (Ledger, Queues, Caching)"]
S6 --> S7["7. Concurrency & Consistency (Pessimistic Locks, Sagas)"]
S7 --> S8["8. Resiliency & Failure Modes (Circuit Breakers, DLQs)"]
S8 --> S9["9. Security & Compliance (PCI-DSS, RBI Localization, HSM)"]
S9 --> S10["10. Observability & Trade-offs Defense"]
```

Step 1: Clarify Requirements

Never start drawing architecture without locking down scope: - **Functional Requirements:** What must the system do? (e.g., User initiates transfer, funds are debited from A and credited to B, audit trail

recorded, notifications dispatched). - **Non-Functional Requirements:** - **Availability vs Consistency (CAP):** Banking systems are **CP systems** (Consistency & Partition Tolerance over Availability). Money cannot be double-spent. - **Latency SLA:** $< 500 \text{ ms}$ for p99 API response. - **Durability:** Zero data loss; 100% auditability.

Step 2: Scale & Capacity Estimation

- **Traffic:** Peak Transactions Per Second (TPS). Example: 10,000 TPS peak during Diwali sales.
- **Storage:** $10,000 \text{ txns/sec} \times 86,400 \text{ sec/day} \approx 864 \text{ million txns/day}$.
At 500 bytes per ledger record: $\approx 432 \text{ GB/day}$ implies $\approx 150 \text{ TB/year}$.
Architectural Conclusion: Single database node will fail; requires horizontal sharding by `account_id` and cold-data archival to object storage (Cloud Storage / S3).

4. Simple Example: Back-of-the-Envelope Calculation Template

Scale Template for IDFC Systems:

- Daily Active Users (DAU): 10 Million
- Average transactions per user per day: 2
- Total daily transactions: 20 Million
- Average TPS = $20\text{M} / 86,400 \approx 230 \text{ TPS}$
- Peak TPS (5x multiplier for festival bursts): $230 * 5 \approx 1,150 \text{ TPS}$
- Storage per record: ~1 KB (PII, tokens, signatures, audit data)
- Daily storage: $20\text{M} * 1 \text{ KB} = 20 \text{ GB/day}$
- 5-Year Regulatory Archive: $20 \text{ GB} * 365 * 5 \approx 36.5 \text{ TB}$

5. Real-World Banking Example: The High-Level Architecture

```

flowchart TD
    Client["Client (Mobile App / Netbanking)"] --> CDN["Cloud Armor / WAF (DDoS Shield)"]
    CDN --> LB["Layer 7 Load Balancer (ALB / GCP HTTPS LB)"]
    LB --> GW["API Gateway / BFF (Auth, Rate Limiting, TLS Termination)"]

    subgraph Microservices["Kubernetes Cluster (Private VPC)"]
        GW --> AuthSvc["Auth & Token Service"]
        GW --> TransferSvc["Transfer Orchestration Service"]
        TransferSvc --> AccSvc["Account & Ledger Service"]
        TransferSvc --> SwitchSvc["Payment Switch Adapter (NPCI/Visa)"]
    end

    subgraph DataTier["Data & Event Tier"]
        AccSvc --> RedisCluster["Redis Cluster (Distributed Locks & Idempotency)"]
        AccSvc --> PrimaryDB["PostgreSQL Primary (Write Master - Multi-AZ)"]
        PrimaryDB --> ReadReplica["PostgreSQL Read Replicas"]
        TransferSvc --> Kafka["Apache Kafka (Ordered Financial Event Stream)"]
        Kafka --> AuditSvc["Audit & Fraud Detection Service"]
        Kafka --> NotifySvc["Transactional Notification Service"]
    end

```

6. Code: Distributed Token Bucket Rate Limiter with Redis Lua Script

```
-- token_bucket.lua
-- Atomically checks and decrements available tokens in Redis
local key = KEYS[1]
local capacity = tonumber(ARGV[1])
local refill_rate = tonumber(ARGV[2]) -- tokens per second
local now = tonumber(ARGV[3])
local requested = tonumber(ARGV[4])

-- Get bucket state: [tokens, last_refreshed_timestamp]
local data = redis.call("HMGET", key, "tokens", "last_updated")
local tokens = tonumber(data[1])
local last_updated = tonumber(data[2])

if not tokens then
    tokens = capacity
    last_updated = now
else
    local delta = math.max(0, now - last_updated)
    tokens = math.min(capacity, tokens + delta * refill_rate)
    last_updated = now
end

if tokens >= requested then
    tokens = tokens - requested
    redis.call("HMSET", key, "tokens", tokens, "last_updated", last_updated)
    redis.call("EXPIRE", key, 3600)
    return 1 -- Allowed
else
    return 0 -- Rejected (HTTP 429 Too Many Requests)
end
```

7. How It Works Internally: CAP & PACELC Theorems in Banking

- **CAP Theorem:** In the presence of a network partition (\$P\$), a distributed system must choose between Consistency (\$C\$) and Availability (\$A\$). Banking systems choose **Consistency**: if the primary database partition cannot communicate with the secondary node, writes must reject rather than accept divergent balances.
- **PACELC Theorem:** Extends CAP: If there is a Partition (\$P\$), trade off Availability (\$A\$) and Consistency (\$C\$); **Else (\$E\$)**, trade off Latency (\$L\$) and Consistency (\$C\$). In normal operations, banking switches accept higher write latency (synchronous replica replication) to guarantee strict serializable consistency.

8. Common Mistakes

- 1. Dumping Technology Names Without Justification:** Saying *"I will use Kafka and Cassandra because they are scalable"* fails an interview. State *why*: *"I select Kafka for the event stream because it provides partitioned, strictly ordered event logs per `account_id` and consumer group replayability."*
- 2. Ignoring Single Points of Failure (SPOF):** Forgetting to declare database read replicas, multi-AZ active-passive failover, or multi-region disaster recovery (RPO and RTO).
- 3. Omitting Financial Idempotency & Reconciliation:** In a banking system design, if you do not mention idempotency keys and end-of-day reconciliation with NPCI/Visa, your design is incomplete.

9. Performance / Complexity Matrix

Component	Technology Choice	Latency SLA	Availability Target
API Gateway	Kong / Envoy / Express Gateway	$< 10\text{ms}$	99.99%
Distributed Cache	Redis Cluster (In-Memory)	$< 2\text{ms}$	99.95%
Event Stream	Apache Kafka	$< 15\text{ms}$	99.99%
Core Relational DB	PostgreSQL Multi-AZ with WAL	$< 25\text{ms}$	99.999% (Five Nines)
Audit Log Store	OpenSearch / GCP Cloud Storage	Async	99.99999999% (11 Nines)

10. Interview Questions (Easy \$to\$ Medium \$to\$ Hard)

Easy

- Q:** What is the difference between horizontal scaling and vertical scaling?

Medium

- Q:** Explain how you choose between Redis and Memcached for caching banking session states.

Hard

- Q:** Design a distributed ledger that handles 50,000 transactions per second across 100 million accounts while preventing hot-spotting on high-volume merchant accounts (e.g., Amazon or Swiggy receiving 5,000 credits/sec).

11. Follow-up Questions from Interviewer

- "If a merchant account receives 5,000 transfers per second, row-level locking on that single merchant `account_id` will serialize and bottleneck the entire database. How do you solve hot-spot account write contention?" (Answer: Sharded balance accounts / Bank Sub-accounts: Split the merchant's balance into 50 sub-accounts in the database. Inbound credits randomly distribute across sub-accounts without locking contention; total balance is the sum of sub-accounts).
 - "What is the difference between RPO (Recovery Point Objective) and RTO (Recovery Time Objective) in disaster recovery planning?"
-

12. Model Answer: PACELC & Synchronous DB Replication in FinTech

Interviewer: "Why don't we use asynchronous database replication for the primary financial ledger?"

Model Answer: "In high-throughput web systems, asynchronous database replication is standard because it maximizes write throughput by allowing the primary node to acknowledge commits before replicas receive the WAL stream.

However, in a banking ledger governed by the **PACELC theorem (PC/EC)**, asynchronous replication introduces the risk of data loss: - If the primary database crashes before WAL records are replicated to the standby replica, the promoted replica suffers from **unreplicated transactions**. - In a banking system, that loss represents money that was debited from a customer or credited to a merchant that has permanently vanished from the failover node (a catastrophic non-zero **Recovery Point Objective (RPO)**).

Therefore, core banking ledgers mandate **Semi-Synchronous or Synchronous Multi-AZ Replication** (such as PostgreSQL with `synchronous_commit = on` and at least one synchronous standby). The primary blocks until at least one independent replica has flushed the WAL to disk, guaranteeing zero data loss (RPO = 0) in the event of an automated primary node failover."

13. Practical Exercise

Review the 10-step template and practice driving a 5-minute verbal walkthrough of designing a **High-Priority OTP Notification Engine** (SMS, WhatsApp, Push) that must deliver OTPs within 3 seconds.

14. Quick Revision

- Follow the 10-step framework: Requirements $\rightarrow$ Scale $\rightarrow$ API $\rightarrow$ Data $\rightarrow$ Architecture $\rightarrow$ Deep Dive $\rightarrow$ Consistency $\rightarrow$ Failure $\rightarrow$ Security $\rightarrow$ Observability.
 - Banking systems are CP systems (Consistency > Availability).
 - Shard databases by `account_id` or `customer_id` to distribute load evenly.
 - Mitigate hot merchant accounts using distributed sub-account balance sharding.
 - Synchronous WAL replication guarantees zero financial data loss (RPO = 0).
-

15. Interview Checklist

- ☐ Memorizes the 10-step system design sequence.
- ☐ Confidently calculates Peak TPS, Storage, and Memory requirements.
- ☐ Explains CAP and PACELC in the context of banking consistency.
- ☐ Solves the merchant hot-spot account contention bottleneck.

System Design: High-Throughput Payment Switch & Settlement Engine

1. Why This Matters

Designing a Payment System is the quintessential System Design interview question at IDFC FIRST Bank. The Strategic Projects team builds payment gateways, card switches, and UPI acquiring engines. An interviewer will push you deep into every architectural layer: from HTTP idempotency keys at the API gateway down to double-entry ledger database tables, Kafka dead-letter queues, and end-of-day bank reconciliation.

2. Prerequisites

- [06-SYSTEM-DESIGN/system-design-fundamentals.md](#)
 - [11-BANKING-FINTECH/transaction-processing.md](#)
 - Relational ACID transactions and distributed Sagas.
-

3. Concept: The Payment System Architecture

```

flowchart TD
    MerchantApp["Merchant App / Client"] --> WAF["Cloud Armor / WAF (DDoS Shield)"]
    WAF --> GW["API Gateway (Rate Limiter + Idempotency Filter)"]

    subgraph PaymentPlatform["IDFC FIRST Bank Payment Platform"]
        GW --> PaySvc["Payment Orchestration Service"]
        PaySvc --> RiskSvc["Risk & Fraud Engine (Velocity Checks)"]
        PaySvc --> LedgerSvc["Double-Entry Ledger Service"]
        PaySvc --> RoutingSvc["Smart Payment Router"]
    end

    subgraph ExternalRails["External Payment Rails"]
        RoutingSvc --> NPCI["NPCI UPI Switch"]
        RoutingSvc --> VisaMC["Visa / Mastercard Gateway"]
        RoutingSvc --> CBS["Core Banking Solution (IDFC CBS)"]
    end

    subgraph AsyncTier["Asynchronous Processing Tier"]
        PaySvc --> Kafka["Apache Kafka (Payment Events Topic)"]
        Kafka --> ReconEngine["Reconciliation & Settlement Engine"]
        Kafka --> WebhookSvc["Merchant Webhook Notification Engine"]
        Kafka --> Analytics["Fraud & Regulatory Reporting"]
    end

```

4. Requirements & Scale Estimation

Functional Requirements

1. **Execute Payment:** Merchant/User submits payment request with payment method (Card, UPI, Netbanking).
2. **Idempotency:** Guarantee that network retries or duplicate button clicks never cause duplicate debits.
3. **Ledger Integrity:** Maintain immutable double-entry records for all financial events.
4. **Reconciliation:** Match internal transaction state with daily settlement clearing files from NPCI/Visa.

Non-Functional Requirements

- **Consistency:** Strict consistency for balance deductions; zero double-spending.
- **Throughput:** Handle 5,000 peak TPS with $< 1.5\text{s}$ p99 latency.
- **Availability:** 99.999% (Five Nines).
- **Security:** PCI-DSS Level 1 compliant; zero raw card data in application logs.

5. API Design & Data Schema

Core API Endpoint

POST /v1/payments/charges

Headers:

Content-Type: application/json

Authorization: Bearer <api_key>

X-Idempotency-Key: <uuid_v4>

Request Body:

```
{
  "amount": 250000,           // In smallest currency unit (2500.00 INR = 250000
                             paise)
  "currency": "INR",
  "customer_id": "cust_89123",
  "payment_method": {
    "type": "UPI",
    "vpa": "aarav@idfcbank"
  },
  "metadata": {
    "order_id": "ORD_991823"
  }
}
```

Relational Database Schema (PostgreSQL)

```
-- Payments Table (Order state machine)
CREATE TABLE payments (
  payment_id VARCHAR(36) PRIMARY KEY,
  idempotency_key VARCHAR(64) UNIQUE NOT NULL,
  customer_id VARCHAR(36) NOT NULL,
  amount BIGINT NOT NULL CHECK (amount > 0), -- Stored in paise
  currency VARCHAR(3) NOT NULL DEFAULT 'INR',
  status VARCHAR(20) NOT NULL CHECK (status IN ('CREATED', 'INITIATED', 'SUCCESS',
'FAILED', 'REFUNDED')),
  payment_channel VARCHAR(20) NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Ledger Entries (Double-Entry Bookkeeping)
CREATE TABLE ledger_entries (
  entry_id BIGSERIAL PRIMARY KEY,
  payment_id VARCHAR(36) NOT NULL,
  account_id VARCHAR(36) NOT NULL,
  direction VARCHAR(6) NOT NULL CHECK (direction IN ('DEBIT', 'CREDIT')),
  amount BIGINT NOT NULL CHECK (amount > 0),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (payment_id) REFERENCES payments(payment_id)
);

CREATE INDEX idx_payments_idempotency ON payments(idempotency_key);
CREATE INDEX idx_ledger_account ON ledger_entries(account_id, created_at);
```

6. Code: Payment State Machine & Idempotency Transition in Node.js

```

const PaymentStatus = {
  CREATED: 'CREATED',
  INITIATED: 'INITIATED',
  SUCCESS: 'SUCCESS',
  FAILED: 'FAILED'
};

class PaymentStateMachine {
  static isValidTransition(current, next) {
    const allowed = {
      [PaymentStatus.CREATED]: [PaymentStatus.INITIATED,
PaymentStatus.FAILED],
      [PaymentStatus.INITIATED]: [PaymentStatus.SUCCESS,
PaymentStatus.FAILED],
      [PaymentStatus.SUCCESS]: [], // Terminal state
      [PaymentStatus.FAILED]: []   // Terminal state
    };
    return allowed[current]?.includes(next) || false;
  }
}

async function updatePaymentStatus(paymentId, fromStatus, toStatus, dbClient) {
  if (!PaymentStateMachine.isValidTransition(fromStatus, toStatus)) {
    throw new Error(`Invalid state transition from ${fromStatus} to
${toStatus}`);
  }

  // Atomic conditional status update
  const result = await dbClient.query(
    `UPDATE payments
     SET status = $1, updated_at = NOW()
     WHERE payment_id = $2 AND status = $3`,
    [toStatus, paymentId, fromStatus]
  );

  if (result.rowCount === 0) {
    throw new Error('Concurrent state transition conflict. Aborting.');
```

7. How It Works Internally: The 3-Way Reconciliation Engine

```

flowchart TD
    subgraph ThreeWayMatch["Daily 3-Way Reconciliation Matching Engine"]
        Internal["1. Internal Ledger\n(Payment Orchestration DB)"]
        SwitchFile["2. Payment Gateway File\n(NPCI / Visa Raw Settlement File)"]
        BankCBS["3. Bank CBS Settlement\n(IDFC Core Banking Statement)"]

        Internal --> Engine["Daily Batch Matching Engine (Spark / Spring Batch)"]
        SwitchFile --> Engine
        BankCBS --> Engine

        Engine --> Matched["Matched & Settled ($100%)"]
        Engine --> Discrepancy["Discrepancy Queue (Investigate: One-Way Debit)"]
    end

```

Reconciliation Workflow:

1. Every night at 02:00 AM, the bank receives **Clearing and Settlement raw files** (e.g., NPCI `EOD_SETTLEMENT.txt`).
2. A distributed batch processor (Apache Spark or Spring Batch) parses the millions of records.
3. For every transaction reference, it checks:
4. Did the internal ledger record `SUCCESS` ?
5. Did NPCI confirm `SUCCESS` ?
6. Did the inter-bank nodal settlement credit the bank's clearing account?
7. Any mismatch triggers automated correction workflows (e.g., auto-reversals or credit adjustments) in accordance with RBI timelines.

8. Common Mistakes

1. **Storing Currency in Decimals/Floats:** Floats create IEEE rounding errors. Always store currency as **64-bit integer paise/cents** (`amount: 50000` = ₹500.00).
2. **Synchronous Webhooks:** Making synchronous HTTP calls to merchant webhooks inside the payment processing request blocks worker threads. Always push webhook events to **Kafka** and let dedicated asynchronous workers dispatch them with exponential backoff.
3. **Omitting Distributed Locking for Balances:** If two payments hit the same account concurrently, failing to lock or verify versions causes balance double-spending.

9. Performance / Complexity Matrix

Layer	Technology	Latency	Scalability Strategy
Idempotency Gate	Redis <code>SET NX</code>	$< 2\text{ms}$	Sharded Redis Cluster
Payment Router	Node.js Microservice	$< 15\text{ms}$	Horizontal Pod Autoscaling (HPA)
Transaction DB	PostgreSQL Multi-AZ	$< 25\text{ms}$	Partitioned by Date / Sharded by Customer ID
Asynchronous Bus	Apache Kafka	$< 10\text{ms}$	Partitioned by <code>customer_id</code> for in-order events

10. Interview Questions (Easy \$to\$ Medium \$to\$ Hard)

Easy

- **Q:** Why should monetary amounts be stored as integers (e.g., paise) rather than floating-point numbers in a payment database?

Medium

- **Q:** How do you guarantee that a merchant webhook notification is reliably delivered even if the merchant's server is down for 6 hours?

Hard

- **Q:** A payment switch experiences a network split while waiting for a response from NPCI. The client is waiting on an open HTTP connection. How should the switch handle the client response, the pending database record, and subsequent reconciliation?

11. Follow-up Questions from Interviewer

- "What is the Dead Letter Queue (DLQ) strategy for failed webhook delivery retries?" (Answer: Retry with exponential backoff: 5s, 30s, 5m, 30m, 2h. After max retries, move to DLQ and trigger email alert to merchant technical contact).
- "How do you partition a Kafka topic for payment events to ensure in-order processing of transactions for a single account?" (Answer: Use `account_id` as the Kafka message partition key. Kafka guarantees strict chronological order within a single partition).

12. Model Answer: Network Split During Payment Gateway Call

Interviewer: "If the payment service sends a debit request to the external banking switch, but the connection times out before receiving a response, what do you return to the user?"

Model Answer: "When an external payment switch call times out, the transaction is in an **IN-DOUBT / PENDING** state. We must NEVER assume failure and refund, nor assume success and fulfill the order.

1. **Immediate API Response:** We return HTTP 202 Accepted with status **PENDING** and the transaction reference to the client, displaying a 'Payment Processing' state.
2. **Active Status Polling:** A background worker immediately initiates **Status Inquiry APIs** to the external switch at 5s, 15s, and 30s intervals using the unique RRN.
3. **Terminal State Resolution:** If the switch confirms the debit succeeded, we transition the payment to **SUCCESS**, write double-entry ledger records, and dispatch a webhook. If confirmed failed, we transition to **FAILED**.
4. **Nightly Reconciliation Fallback:** If the status inquiry remains unresponsive, the transaction remains **PENDING** until the nightly settlement file resolves it with 100% finality."

13. Practical Exercise

Draw the high-level architecture diagram of a Payment Gateway on a whiteboard or paper, ensuring you include: API Gateway, Redis Idempotency Cache, Payment Orchestrator, PostgreSQL Ledger DB, Kafka Event Stream, and the Reconciliation Batch Worker.

14. Quick Revision

- Currency amounts must always be stored in smallest units (integer paise).
- Idempotency middleware intercepts requests via **X-Idempotency-Key** and Redis **SET NX**.
- Ledger entries are immutable: every transfer creates balanced Debits and Credits.
- Sagas coordinate distributed transaction steps with compensating reversals.
- End-of-day reconciliation guarantees that internal ledgers match bank switch clearing files.

15. Interview Checklist

- [] Draws the complete payment flow architecture confidently.

- [] Defines the relational schema with double-entry ledger entries.
- [] Explains the state machine transitions and prevents race conditions.
- [] Explains 3-way reconciliation against NPCI/Visa clearing files.

System Design Last-Minute Interview Cheatsheet

Numbers Every Engineer Must Know (Latencies)

L1 Cache Reference:	~0.5 ns
Branch Mispredict:	~5 ns
L2 Cache Reference:	~7 ns
Mutex Lock/Unlock:	~25 ns
Main Memory (RAM) Reference:	~100 ns
Compress 1KB with Snappy:	~3 µs
Send 1KB over 1Gbps Network:	~10 µs
Read 1MB Sequentially from SSD:	~1 ms
Disk Seek (HDD):	~10 ms
Send Packet Across Datacenter:	~0.5 ms
Round Trip (Mumbai to US-East):	~150 ms

High Availability & Uptime SLAs

Availability Target	Annual Allowed Downtime	Daily Allowed Downtime	Typical Tier
99% ("Two Nines")	3.65 days	14.4 minutes	Internal testing tools
99.9% ("Three Nines")	8.76 hours	43.2 seconds	Standard SaaS apps
99.99% ("Four Nines")	52.6 minutes	4.32 seconds	High-priority API gateways
99.999% ("Five Nines")	5.26 minutes	0.43 seconds	Core Banking Solution & Payment Rails

Redis Caching Strategies & Defenses

- 1. Cache-Aside (Lazy Loading):** App reads cache. On miss, reads DB, writes to cache, returns. (Most common).
- 2. Write-Through:** App writes to cache; cache synchronously writes to DB before acknowledging.

3. **Write-Behind (Write-Back):** App writes to cache; cache asynchronously batches writes to DB.
(High write throughput, risk of data loss on crash).
 4. **Cache Penetration Defense:** Querying keys that don't exist in DB.
 5. *Fix:* Cache `null` with short TTL, or use a **Bloom Filter** at the gateway.
 6. **Cache Stampede / Breakdown:** Hot key with high QPS expires; thousands of requests hit DB simultaneously.
 7. *Fix:* Mutual exclusion lock (Mutex in Redis) or probabilistic early expiration (XFetch algorithm).
-

Apache Kafka Quick Architecture

- **Topic:** Logical stream of messages.
- **Partition:** Ordered, append-only log of immutable messages. The unit of parallelism.
- **Partition Key:** Messages with the same key (e.g., `account_id`) **always go to the same partition**, guaranteeing chronological ordering per account.
- **Consumer Group:** Multiple workers reading a topic. Each partition is consumed by strictly **one** consumer within a group.
- **Replication Factor:** `N` copies across brokers (typical in banking: 3 replicas with `min.insync.replicas = 2` and `acks = all` for zero message loss).